
Introdução a programação com Python

Algoritmos e Técnicas de Programação

Professor Wendell Bento Geraldes

Instituto Federal de Goiás
campus Luziânia
<wendell.geraldes@ifg.edu.br>

2024



- 1 Manipulação de arquivos com Python
- 2 Trabalhando com Arquivos de Texto vs. Arquivos Binários
- 3 Exercícios



Manipulação de arquivos com Python



Manipulando arquivos com Python

Manipulação de arquivos

Em informática é necessário possuir conhecimentos em manipulação de arquivos, seja para armazenar os resultados de um determinado programa ou ler informações já armazenadas em arquivos. Independentemente do domínio de aplicação – seja em desenvolvimento web, análise de dados, automação ou qualquer outra área – a capacidade de ler, escrever e manipular arquivos é uma ferramenta poderosa no arsenal de qualquer desenvolvedor.

No Python

O Python possui funções previamente definidas que servem para criar, ler, atualizar e remover arquivos.



Manipulando arquivos com Python

Abrindo arquivos

Para iniciar qualquer operação com arquivos em Python, precisamos primeiro abrir o arquivo desejado. Isso é feito utilizando a função incorporada **open()**. Esta função necessita de pelo menos um argumento – *o caminho para o arquivo que desejamos abrir*. Além disso, podemos especificar o modo de operação, que determina se queremos ler o arquivo ('r'), escrever nele ('w'), acrescentar dados ('a'), entre outros (??).

Parâmetros da função open()

- r - Ler - Este é o parâmetro padrão. Serve para ler um arquivo. Caso ele não exista, uma mensagem de erro aparece.
- w - Escreve - Abre o arquivo para escrita. Este parâmetro cria o arquivo se ele ainda não existir.
- a - Adição - Abre o arquivo para escrita. Cria o arquivo se ele não existir e se existir adiciona um registro no final.



Escrevendo em arquivos com Python

Escrever em arquivos é tão fundamental quanto lê-los. Seja para registrar dados de um experimento, atualizar um arquivo de configuração ou simplesmente criar um novo documento de texto, Python simplifica o processo de escrita de forma notável. Para escrever em um arquivo, primeiro precisamos abri-lo em modo de escrita. Isso é feito usando **open()** com o modo **'w'** para escrita (que sobrescreve o arquivo se ele já existir) ou **'a'** para anexar (que mantém o conteúdo existente e adiciona novos dados no fim) (**??**). Veja o exemplo a seguir:



Manipulando arquivos com Python

Exemplos

```
# Aqui vamos criar o arquivo exemplo.txt usando a função open(), lembre que para escrever  
# no arquivo usamos o parametro w  
arquivo = open('exemplo.txt', 'w')  
# Aqui vamos escrever uma mensagem no arquivo usando o método arquivo.write()  
arquivo.write('Escrevendo no arquivo pela primeira vez.')  
# Fechar o arquivo é muito importante, por isso usamos o método arquivo.close()  
arquivo.close()
```



Leitura de arquivos com Python

Após abrir um arquivo, o próximo passo natural é ler o seu conteúdo. Python oferece várias maneiras de fazer isso, adequadas para diferentes necessidades. A forma mais simples é utilizando o método **.read()** para ler todo o conteúdo de uma vez. No entanto, para arquivos grandes, essa abordagem pode não ser eficiente ou mesmo viável devido à limitação de memória. Para arquivos maiores, uma abordagem melhor é ler o arquivo linha por linha, usando um laço. Isso pode ser feito de forma simples (??).



Manipulando arquivos com Python

Exemplos

```
# Primeiro vamos ver como abrir um arquivo pequeno com poucas linhas
arquivo = open('exemplo.txt', 'r')
print(arquivo.read())

# Agora vamos ver como abrir um arquivo grande com muitas linhas
# usando um laço de repetição for
with open('exemplo.txt', 'r') as arquivo:
    for linha in arquivo:
        print(linha)
```



Fechando os arquivos de forma segura

Ao trabalhar com arquivos, é crucial garantir que eles sejam fechados corretamente após a conclusão das operações de leitura ou escrita. Fechar um arquivo com **close()** é fundamental porque:

- Libera recursos do sistema que estavam sendo utilizados pelo arquivo.
- Garante que todas as alterações feitas no arquivo sejam efetivamente salvas.



Exemplos

```
# Abrindo um arquivo para leitura
arquivo = open('exemplo.txt', 'r')
# Lendo uma linha do arquivo e mostrando na tela do computador
print(arquivo.readline())
# Fechando o arquivo
arquivo.close()
```



Usando o gerenciador de contexto

O gerenciador de contexto **with** é uma das ferramentas mais poderosas e elegantes do Python para o trabalho com arquivos. Ao usar **with**, Python cria um contexto temporário onde o arquivo é aberto e garante que ele seja fechado automaticamente ao final do bloco, independente de como esse bloco é encerrado (??).



Manipulando arquivos com Python

Exemplos

```
# Abre o arquivo com gerador de contexto with e coloca em modo de escrita
with open('exemplo.txt', 'w') as arquivo:
# Escreve a mensagem no arquivo e depois fecha automaticamente
    arquivo.write('Usando with para manipular arquivos.')
```



Trabalhando com Arquivos de Texto vs. Arquivos Binários



Trabalhando com Arquivos de Texto vs. Arquivos Binários

Entendendo as diferenças: Arquivos de texto

Os **arquivos de texto** são aqueles que contêm dados que podem ser lidos como texto. Isso inclui documentos .txt, arquivos Python (.py), e arquivos de configuração. Quando trabalhamos com esse tipo de arquivo, Python assume uma codificação de caracteres (geralmente UTF-8) e trata o arquivo de forma que as quebras de linha e caracteres especiais sejam interpretados corretamente (??).

Entendendo as diferenças: Arquivos binários

Já os **arquivos binários** contêm dados que não são destinados à leitura diretamente como texto, como imagens, vídeos, executáveis, entre outros. Quando lidamos com esses arquivos, usamos o modo 'b' em conjunto com os modos de leitura ou escrita ('rb' ou 'wb'), o que indica ao Python que o arquivo deve ser tratado como um fluxo de bytes, sem qualquer interpretação de codificação ou quebras de linha (??).



Trabalhando com Arquivos de Texto vs. Arquivos Binários

Copiar, Mover e Renomear de Forma Eficiente

Para quem trabalha com Python, manipular arquivos – seja copiando, movendo ou renomeando – é uma tarefa do dia a dia que pode ser simplificada com o conhecimento das ferramentas certas. Utilizando o módulo **shutil**, por exemplo, podemos fazer todas essas operações com facilidade sem ter que recorrer a código de baixo nível (??).



Trabalhando com Arquivos de Texto vs. Arquivos Binários

Copiar um arquivo

Para copiar um arquivo, usaríamos a função **shutil.copy(source, destination)**, onde **source** é o caminho do arquivo original e **destination** pode ser um diretório (onde o arquivo será copiado com o mesmo nome) ou um caminho completo incluindo um novo nome de arquivo.

Mover um arquivo

Para mover um arquivo, a função **shutil.move(source, destination)** opera de forma similar, com a diferença de que o arquivo original não permanece no local de origem.

Renomear um arquivo

Já para renomear, o módulo **os** oferece a função **os.rename(source, destination)**, que também pode ser usada para mover arquivos, mas sem a flexibilidade adicional de **shutil** para lidar com diferentes sistemas de arquivos e permissões (??).



Trabalhando com Arquivos de Texto vs. Arquivos Binários

Exemplos de uso do módulo shutil

```
# Em primeiro lugar precisamos importar a biblioteca shutil para o programa
import shutil
# shutil.copy(src, dst) serve para copiar um arquivo de um lugar para outro
shutil.copy('arquivo.txt', 'caminho/destino/arquivo_copiado.txt')
# shutil.move(src, dst) serve para mover um arquivo de um lugar para outro
shutil.move('arquivo.txt', 'novo_lugar/arquivo_movido.txt')
# shutil.rmtree(path) serve para remover uma pasta e todo o seu
# conteúdo de forma recursiva
shutil.rmtree('nome_da_pasta_que_vai_ser_removida')
```



Exercícios



Cadastro de Contatos

Objetivo: Criar um programa que permita ao usuário adicionar e visualizar contatos em um arquivo de texto.

- 1 Crie um programa que permita ao usuário:
 - Adicionar um novo contato (nome, telefone e e-mail).
 - Verificar os contatos já cadastrados.
- 2 O programa deve:
 - Salvar os contatos em um arquivo chamado contatos.txt.
 - Ao adicionar um novo contato, o programa deve escrever os dados no final do arquivo.
 - Ao visualizar os contatos, o programa deve ler o conteúdo do arquivo e mostrar todos os contatos cadastrados.



Passos

- 1 Crie um arquivo chamado contatos.txt (caso não exista) para armazenar os dados.
- 2 Crie uma função para adicionar um novo contato.
- 3 Crie uma função para visualizar todos os contatos salvos.
- 4 No menu, ofereça a opção de adicionar ou visualizar os contatos.
- 5 Use manipulação básica de arquivos (leitura e escrita).



 SCIENCE, D. T. I. A. . D. **Abrindo e manipulando arquivos com Python: aprenda agora!** DIDATICA TECH, 2025. Disponível em: <<https://didatica.tech/abrindo-e-manipulando-arquivos-com-python-aprenda-agora/>>.

